

Proyecto Final: Triage de tickets de soporte con AI

1. Introducción

Este documento describe la estructura de datos del sistema de triage de tickets de soporte, incluyendo:

- El esquema de las tablas vinculadas en Airtable
- Los esquemas JSON de transferencia entre las integraciones
- El flujo de datos completo desde el trigger hasta el almacenamiento

El sistema está compuesto por dos tablas relacionadas (Tickets y Clientes) y utiliza Make como orquestador, OpenAI como motor de IA, y Slack como canal de salida.

2. Esquema de la Base de Datos en Airtable

2.1. Tabla Tickets

Campo	Tipo	Descripción	¿Quién lo completa?
Nombre	Single text line	Nombre del cliente que envía el ticket	Gmail trigger
Email	Email	correo electrónico del cliente	Gmail trigger
Mensaje	Long text	Contenido del correo	Gmail trigger
Prioridad	Single select	Clasificación según la IA	Parse JSON
Estado	Single select	Estado actual del ticket en el pipeline	Flujo 1 al principio, flujo 2 al terminar
Fecha	Date/Time	Fecha de recepción del ticket	Automático
Aprobado	Checkbox	Validación humana	Humano
Respuesta AI	Long text	Respuesta sugerida por la IA	Parse JSON

Cliente	Link to another record	Relacion con tabla Clientes	Manual o Automático
Última modificación	Last modified time	Campo para detectar cambios (importante para flujo 2)	Automático

2.2. Tabla Clientes

Campo	Tipo	Descripción
Nombre	Single text line	Nombre del cliente
Email	Email	Correo electronico del cliente
Historial	Long text	Notas sobre interacciones previas
Tickets	Link to another record	Relación inversa con tabla Tickets
Cantidad de Tickets	Count	1. Cuantos tickets tiene el cliente

2.3. Relación tabla Tickets y tabla Clientes

- Un Cliente puede tener muchos Tickets
- Un Ticket pertenece a un solo Cliente
- La relación se gestiona mediante el campo Cliente en la tabla Tickets

3. Esquemas JSON de Transferencia

3.1 JSON de OpenAI hacia Parse JSON

La IA devuelve un JSON con la clasificación y la respuesta sugerida:

```
json
{
  "prioridad": "Alta",
  "respuesta_sugerida": "Estimado cliente, lamento
```

```
mucho lo sucedido con su pedido...",  
  "razon": "El cliente reporta un problema crítico  
con su pedido y solicita solución inmediata"  
}
```

Explicación de campos:

Campo	Tipo	Descripción	Uso en el flujo
Prioridad	String	Clasificación de urgencia	Usado por el Router para enrutar
respuesta_sugerida	String	Texto de respuesta al cliente	Guardado en campo Borrador
Razón	String	Justificación de la clasificación	Auditoría del equipo

3.2 JSON de Make hacia Airtable

Make envía los datos a Airtable con variables dinámicas:

```
json  
{  
  "Nombre": "{{trigger.from.name}}",  
  "Email": "{{trigger.from.email}}",  
  "Mensaje": "{{trigger.text}}",  
  "Prioridad": "{{parseJSON.prioridad}}",  
  "Estado": "Procesado por IA",  
  "Borrador": "{{parseJSON.respuesta_sugerida}}",  
  "Aprobado": false  
}
```

Explicación de mapeo:

Campo Airtable	Variable de Make	Origen
----------------	------------------	--------

Nombre	{{trigger.from.name}}	Gmail Trigger
Email	{{trigger.from.email}}	Gmail Trigger
Mensaje	{{trigger.text}}	Gmail Trigger
Prioridad	{{parseJSON.prioridad}}	Parse JSON (OpenAI)
Estado	"Procesado por IA"	Texto fijo
Borrador	{{parseJSON.respuesta_sugerida}}	Parse JSON (OpenAI)
Aprobado	false	Valor por defecto

3.3 JSON de Airtable Trigger hacia Gmail (Flujo 2)

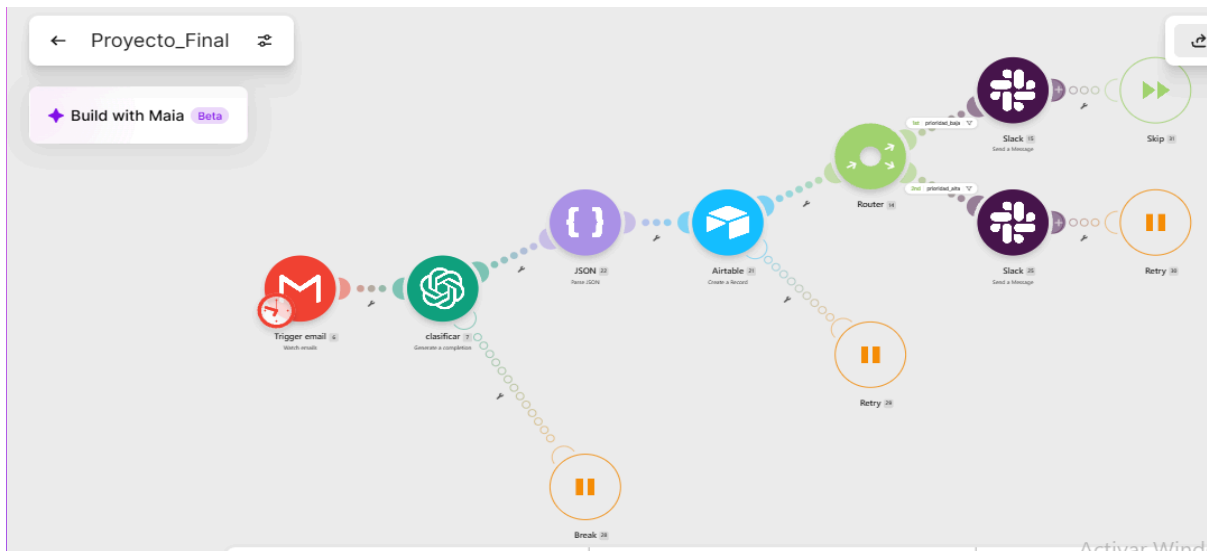
El Flujo 2 detecta cuando un ticket fue aprobado y envía la respuesta:

```
json
{
  "To": "{{airtable.Email}}",
  "Subject": "Respuesta a tu ticket: {{airtable.Nombre}}",
  "Content": "{{airtable.Borrador}}"
}
```

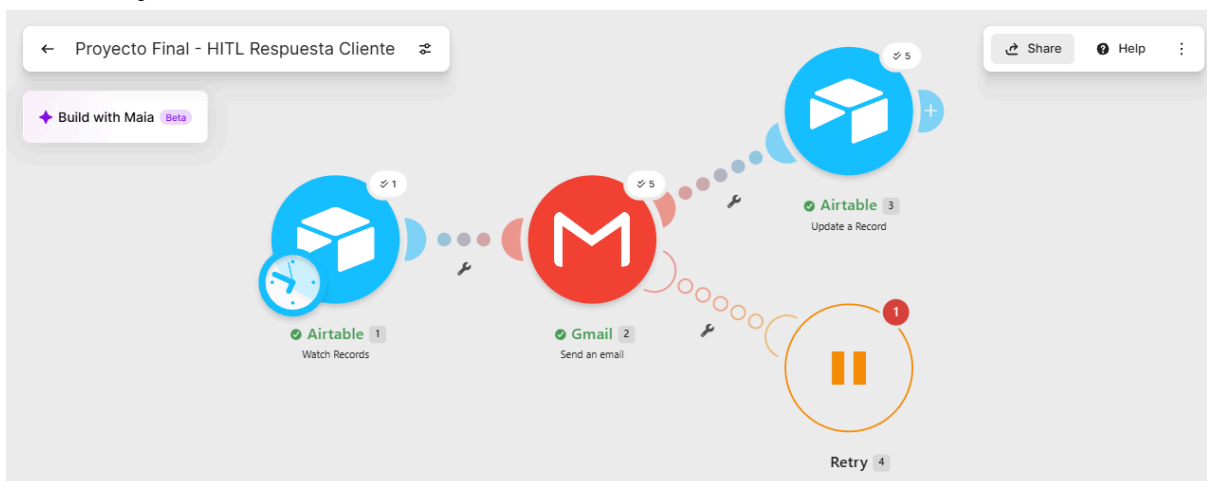
4. Flujo de datos completo

4.1 Flujos visuales

Flujo 1:



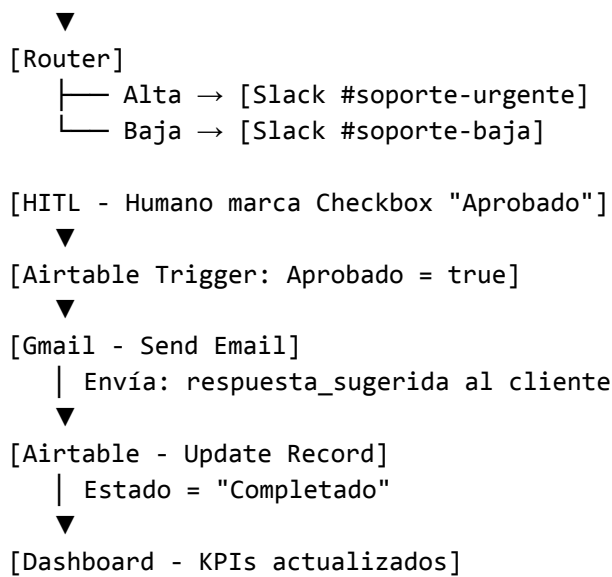
Flujo 2:



4.2 Flujo ilustrado

```

[Gmail]
  | Extrae: from.name, from.email, text
  ▼
[OpenAI]
  | Genera: {prioridad, respuesta_sugerida, razon}
  ▼
[Parse JSON]
  | Separa: prioridad, respuesta_sugerida, razon
  ▼
[Airtable - Create Record]
  | Guarda: Nombre, Email, Mensaje, Prioridad, Estado, Borrador
  | Estado = "Procesado por IA"
  
```



Extra: Aquí no hay datos Hardcodeados, solo variables dinámicas que viajan.

5. Validación de datos

Tipo de validación	Dónde	Qué valida
Campos obligatorios	Gmail Trigger	Nombre y Email vienen del remitente
Validación de IA	OpenAI	Prioridad solo acepta "Alta" o "Baja"
Validación humana	Checkbox Aprobado	Solo el humano puede aprobar
Validación de estado	Flujo 2	El estado cambia a "Completado" solo después de la aprobación

6. Minimización de datos

Los únicos datos del cliente que se guardan que son fundamentales son:

Dato	¿Se guarda?	¿Por qué?
Nombre	☒ Sí	Para personalizar la respuesta

Email	☒ Sí	Para responder al cliente
Mensaje	☒ Sí	Para entender la consulta

7. Conclusión

En resumen, este manual deja documentado cómo está armado el sistema de datos del proyecto: las dos tablas de Airtable (Tickets y Clientes) con su relación, los JSON que se pasan entre OpenAI, Make y Airtable, y el recorrido completo que hace cada dato desde que llega el correo hasta que se guarda y se responde.

La idea principal fue guardar solo lo necesario. No hay datos de más ni información sensible dando vueltas, y cada campo tiene una razón de ser dentro del flujo. Eso hace que el sistema sea más simple de mantener y más seguro.

Con esto queda cerrada la parte de estructuras de datos del proyecto. El resto de los entregables (diagrama, costos, seguridad y dashboard) complementan esta documentación.